



P P SAVANI
UNIVERSITY



Institute of
Computer Application

LABORATORY MANUAL

Department of CS-IT

Cryptography and Network Security
(SSCS4020)

Name of Student: **Shah Rishabh Alpeshbhai**

Enrolment No.: **23SS02IT181** Academic Year: **2026-27**



P P SAVANI UNIVERSITY
INSTITUTE OF COMPUTER SCIENCE & APPLICATION

CERTIFICATE

This is to certify that

Mr./Ms. **Shah Rishabh Alpeshbhai**

Of **Bachelor of Science Honours in Information Technology**

Engineering having Enrollment No. **23SS02IT181** has completed
his/her Term work in the subject of

Cryptography & Network Security

(SSCS4020)

Marks Obtained: _____ out of _____

Sign of Faculty

Date: _____

Sign of Head of Department

Date: _____

Contents

Sr. No.	Practical	Page No.	Marks	Date	Sign
01.	Classical Encryption Techniques – Caesar Cipher, Monoalphabetic Cipher, Playfair Cipher and Vigenère Cipher	05		01-07-2026	
02.	Implementation of Playfair Cipher	07		01-07-2026	
03.	Implementation of Hill Cipher	10		08-07-2026	
04.	Implementation of Vigenère Cipher	14		20-07-2026	
05.	Implementation of DES Encryption using Python	17		23-07-2026	
06.	Implementation of RSA Algorithm – Encryption and Decryption	20		19-08-2026	
07.	Case Studies on Message Authentication, Hash Functions, Digital Signatures & DSS	23		25-09-2026	
08.	Diffie-Hellman Key Exchange, Digital Signature, PGP, Cyberattack Case Study & Steganography	28		28-09-2026	

Rules for Computer Laboratory

1. Always keep silence in the lab.
2. Clean the working area of the laboratory as clean as possible.
3. Read the allotted practical before implementation.
4. Stay in the laboratory for a complete 2 hours. Engage yourself in learning the practical.
5. Use of mobile phones is strictly prohibited in the laboratory.
6. Every time bring the Lab Manual for Practical.
7. Bring your own device.

Practical No: 01

Problem Statement: 01

Aim: To implement the Caesar Cipher algorithm for encrypting and decrypting a given plaintext using a fixed shift key.

Code:

```
def encrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            shift = (ord(char.upper()) - 65 + key) % 26
            result += chr(shift + 65)
        else:
            result += char

    return result

def decrypt(cipher, key):
    result = ""

    for char in cipher:
        if char.isalpha():
            shift = (ord(char.upper()) - 65 - key) % 26
            result += chr(shift + 65)
        else:
            result += char

    return result

text = input("Enter Plaintext: ")
key = int(input("Enter Key: "))

cipher = encrypt(text, key)
plain = decrypt(cipher, key)

print("Ciphertext:", cipher)
print("Decrypted Text:", plain)
```

Output:

```
Enter Plaintext: Rishabh
Enter Key: 4
Ciphertext: VMWLEFL
Decrypted Text: RISHABH
```

Problem Statement: 02

Aim: To simulate secure message transmission using the Caesar Cipher algorithm with user-defined shift values.

Code:

```
def encrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            shift = (ord(char.upper()) - 65 + key) % 26
            result += chr(shift + 65)
        else:
            result += char

    return result

def decrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            shift = (ord(char.upper()) - 65 - key) % 26
            result += chr(shift + 65)
        else:
            result += char

    return result

message = input("Enter Message: ")
key = int(input("Enter Key: "))

encrypted = encrypt(message, key)
decrypted = decrypt(encrypted, key)

print("Encrypted Message:", encrypted)
print("Decrypted Message:", decrypted)
```

Output:

```
Enter Message: RISHABH
Enter Key: 4
Encrypted Message: VMWLEFL
Decrypted Message: RISHABH
```

Date**Marks****Signature**

Practical No: 02

Problem Statement: 01

Aim: Implementation of Playfair Cipher for Encryption and Decryption Using Python

Code:

```
def key_matrix(key):
    key = key.upper().replace("J", "I")
    matrix = []
    used = set()

    for ch in key + "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
        if ch.isalpha() and ch not in used:
            used.add(ch)
            matrix.append(ch)

    return [matrix[i:i+5] for i in range(0, 25, 5)]

def pos(matrix, ch):
    for i in range(5):
        for j in range(5):
            if matrix[i][j] == ch:
                return i, j

def prepare(text):
    text = text.upper().replace("J", "I")
    text = ''.join(ch for ch in text if ch.isalpha())

    result = ""
    i = 0

    while i < len(text):
        a = text[i]

        if i + 1 < len(text):
            b = text[i + 1]
        else:
            b = "X"

        if a == b:
            result += a + "X"
            i += 1
        else:
```

```

        result += a + b
        i += 2

    return result

def playfair(text, matrix, encrypt=True):
    result = ""
    shift = 1 if encrypt else -1

    for i in range(0, len(text), 2):
        a, b = text[i], text[i + 1]
        r1, c1 = pos(matrix, a)
        r2, c2 = pos(matrix, b)

        if r1 == r2:
            result += matrix[r1][(c1 + shift) % 5]
            result += matrix[r2][(c2 + shift) % 5]

        elif c1 == c2:
            result += matrix[(r1 + shift) % 5][c1]
            result += matrix[(r2 + shift) % 5][c2]

        else:
            result += matrix[r1][c2]
            result += matrix[r2][c1]

    return result

key = input("Enter Keyword: ")
plaintext = input("Enter Plaintext: ")

matrix = key_matrix(key)
prepared_text = prepare(plaintext)

print("\nKey Matrix:")
for row in matrix:
    print(row)

encrypted = playfair(prepared_text, matrix, True)
decrypted = playfair(encrypted, matrix, False)

print("\nEncrypted Text:", encrypted)
print("Decrypted Text:", decrypted)

```

Output:

```
Enter Keyword: RISHABH
Enter Plaintext: TANI

Key Matrix:
['R', 'I', 'S', 'H', 'A']
['B', 'C', 'D', 'E', 'F']
['G', 'K', 'L', 'M', 'N']
['O', 'P', 'Q', 'T', 'U']
['V', 'W', 'X', 'Y', 'Z']

Encrypted Text: UHKA
Decrypted Text: TANI
```

Problem Statement: 02

Aim: Secure communication using Playfair Cipher with a shared keyword.

Code:

```
# Sender
keyword = input("Enter Shared Keyword: ")
message = input("Enter Message: ")

matrix = key_matrix(keyword)
prepared_message = prepare(message)

encrypted_message = playfair(prepared_message, matrix, True)

print("Encrypted Message:", encrypted_message)

# Receiver
decrypted_message = playfair(encrypted_message, matrix, False)

print("Decrypted Message:", decrypted_message)
```

Output:

```
Enter Shared Keyword: RISHABH
Enter Message: TANI
Encrypted Message: UHKA
Decrypted Message: TANI
```

Date

Marks

Signature

Practical No: 03

Problem Statement: 01

Aim: Implementation of Hill Cipher Encryption and Decryption using Matrix Multiplication and Modular Arithmetic

Code:

```
key = [[3, 3],
        [2, 5]]

# Inverse Key Matrix (Modulo 26)
inverse_key = [[15, 17],
               [20, 9]]

# Convert text to numbers
def text_to_numbers(text):
    return [ord(ch) - ord('A') for ch in text]

# Convert numbers to text
def numbers_to_text(numbers):
    return "".join(chr(num + ord('A')) for num in numbers)

# Prepare plaintext
def prepare_text(text):
    text = text.upper().replace(" ", "")
    if len(text) % 2 != 0:
        text += "X"
    return text

# Encryption Function
def encrypt(text):
    text = prepare_text(text)
    nums = text_to_numbers(text)
    result = []

    for i in range(0, len(nums), 2):
        x = nums[i]
        y = nums[i + 1]

        c1 = (key[0][0] * x + key[0][1] * y) % 26
        c2 = (key[1][0] * x + key[1][1] * y) % 26

        result.extend([c1, c2])

    return numbers_to_text(result)
```

```

# Decryption Function
def decrypt(cipher):
    nums = text_to_numbers(cipher)
    result = []

    for i in range(0, len(nums), 2):
        x = nums[i]
        y = nums[i + 1]

        p1 = (inverse_key[0][0] * x + inverse_key[0][1] * y) % 26
        p2 = (inverse_key[1][0] * x + inverse_key[1][1] * y) % 26

        result.extend([p1, p2])

    return numbers_to_text(result)

# Main Program
plaintext = input("Enter Plain Text: ")

ciphertext = encrypt(plaintext)
decrypted = decrypt(ciphertext)

print("\n----- RESULT -----")
print("Plain Text :", prepare_text(plaintext))
print("Encrypted Text :", ciphertext)
print("Decrypted Text :", decrypted)

if prepare_text(plaintext) == decrypted:
    print("Verification : SUCCESS")
else:
    print("Verification : FAILED")

```

Output:

```

Enter Plain Text: SHAH RISHABH

----- RESULT -----
Plain Text : SHAHRISHABHX
Encrypted Text : XTVJXWXTDFMZ
Decrypted Text : SHAHRISHABHX
Verification : SUCCESS

```

Problem Statement: 02

Aim: Student Task Code

Code:

```

key = [[3, 3],
        [2, 5]]

inverse = [[15, 17],
            [20, 9]]

text = "INFORMATION SECURITY"

text = text.upper().replace(" ", "")

# Add X if length is odd
if len(text) % 2 != 0:
    text += "X"

# Encryption
cipher = ""

for i in range(0, len(text), 2):
    a = ord(text[i]) - 65
    b = ord(text[i + 1]) - 65

    c1 = (key[0][0] * a + key[0][1] * b) % 26
    c2 = (key[1][0] * a + key[1][1] * b) % 26

    cipher += chr(c1 + 65)
    cipher += chr(c2 + 65)

plain = ""

for i in range(0, len(cipher), 2):
    a = ord(cipher[i]) - 65
    b = ord(cipher[i + 1]) - 65

    p1 = (inverse[0][0] * a + inverse[0][1] * b) % 26
    p2 = (inverse[1][0] * a + inverse[1][1] * b) % 26

    plain += chr(p1 + 65)
    plain += chr(p2 + 65)

print("Original Plain Text :", "INFORMATION SECURITY")
print("Plain Text :", text)
print("Encrypted Text :", cipher)
print("Decrypted Text :", plain)

if text == plain:
    print("Verification : SUCCESS")
else:
    print("Verification : FAILED")

```

Output:

```
Original Plain Text : INFORMATION SECURITY  
Plain Text : INFORMATIONSECURITYX  
Encrypted Text : LDFCJQFROIPMSSHVDHLH  
Decrypted Text : INFORMATIONSECURITYX  
Verification : SUCCESS
```

Date**Marks****Signature**

Practical No: 04

Problem Statement: 01

Aim: Implementation of Vigenère Cipher using Python.

Code:

```
# Vigenere Cipher - Encryption and Decryption

def generate_key(text, key):
    key = key.upper()
    text = text.upper()

    key_list = list(key)

    while len(key_list) < len(text):
        key_list.extend(list(key))

    return "".join(key_list[:len(text)])

def encrypt(plaintext, key):
    plaintext = plaintext.upper().replace(" ", "")
    key = generate_key(plaintext, key)

    cipher = ""

    for p, k in zip(plaintext, key):
        x = (ord(p) - ord('A') + ord(k) - ord('A')) % 26
        cipher += chr(x + ord('A'))

    return cipher

def decrypt(ciphertext, key):
    ciphertext = ciphertext.upper().replace(" ", "")
    key = generate_key(ciphertext, key)

    plain = ""

    for c, k in zip(ciphertext, key):
        x = (ord(c) - ord('A') - (ord(k) - ord('A')) + 26) % 26
        plain += chr(x + ord('A'))

    return plain
```

Main Program

```
plaintext = input("Enter Plaintext : ")
keyword = input("Enter Keyword : ")

ciphertext = encrypt(plaintext, keyword)

print("\nEncrypted Text :", ciphertext)

decrypted = decrypt(ciphertext, keyword)

print("Decrypted Text :", decrypted)
```

Output:

```
Enter Plaintext : RISHBAH
Enter Keyword : TANI

Encrypted Text : KIFPUAU
Decrypted Text : RISHBAH
```

Problem Statement: 02

Aim: To simulate secure communication using the Vigenère Cipher with a shared secret key.

Code:

```
def generate_key(text, key):
    key = key.upper()
    text = text.upper()

    key_list = list(key)

    while len(key_list) < len(text):
        key_list.extend(list(key))

    return "".join(key_list[:len(text)])

def encrypt(message, key):
    message = message.upper().replace(" ", "")
    key = generate_key(message, key)

    cipher = ""

    for m, k in zip(message, key):
        x = (ord(m) - ord('A') + ord(k) - ord('A')) % 26
        cipher += chr(x + ord('A'))
```

```

    return cipher

def decrypt(cipher, key):
    cipher = cipher.upper().replace(" ", "")
    key = generate_key(cipher, key)

    message = ""

    for c, k in zip(cipher, key):
        x = (ord(c) - ord('A') - (ord(k) - ord('A')) + 26) % 26
        message += chr(x + ord('A'))

    return message

print("----- Sender -----")

message = input("Enter Message : ")
keyword = input("Enter Shared Secret Key : ")

cipher = encrypt(message, keyword)

print("\nCiphertext Sent :", cipher)

print("\n----- Receiver -----")

received = decrypt(cipher, keyword)

print("Received Ciphertext :", cipher)
print("Decrypted Message :", received)

```

Output:

```

----- Sender -----
Enter Message : RISHABH
Enter Shared Secret Key : TANI

Ciphertext Sent : KIFPTBU

----- Receiver -----
Received Ciphertext : KIFPTBU
Decrypted Message : RISHABH

```

Date

Marks

Signature

Practical No: 05

Problem Statement – 1

AIM: To implement the DES (Data Encryption Standard) algorithm for encrypting and decrypting a plaintext message using Python.

Code:

```
!pip install pycryptodome

from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

# Secret key (8 bytes)
key = b'ABCDEFGH'

# Create DES cipher
cipher = DES.new(key, DES.MODE_ECB)

# Input plaintext
plaintext = input("Enter Plaintext: ")

# Convert into bytes
data = plaintext.encode()

# Apply padding
padded_data = pad(data, DES.block_size)

# Encrypt
ciphertext = cipher.encrypt(padded_data)

print("\nEncrypted Ciphertext (Bytes):")
print(ciphertext)

print("\nEncrypted Ciphertext (Hex):")
print(ciphertext.hex())

# Create DES cipher for decryption
decipher = DES.new(key, DES.MODE_ECB)

# Decrypt
decrypted = decipher.decrypt(ciphertext)

# Remove padding
original = unpad(decrypted, DES.block_size)
```

```
print("\nDecrypted Text:", original.decode())
```

Output:

```
Enter Plaintext: SHAH RISHABH

Encrypted Ciphertext (Bytes):
b'\x14Q\xc5\x8c\xe5s)\xef\xc8U\xdf!\xf2\xf8\x9a\x01'

Encrypted Ciphertext (Hex):
1451c58ce57329efc855df21f2f89a01

Decrypted Text: SHAH RISHABH
```

Problem Statement – 2

Aim: To simulate secure communication using DES encryption with a shared secret key.

Code:

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

# Shared Secret Key
key = b'ABCDEFGH'

print(" Sender :")

message = input("Enter Message :→")

cipher = DES.new(key, DES.MODE_ECB)

ciphertext = cipher.encrypt(
    pad(message.encode(), DES.block_size)
)

print("\nCiphertext Sent (Hex):")
print(ciphertext.hex())

print("\n Receiver :→")

decipher = DES.new(key, DES.MODE_ECB)

received = unpad(
    decipher.decrypt(ciphertext),
    DES.block_size
)
```

)

```
print("Received Ciphertext :")  
print(ciphertext.hex())
```

```
print("Decrypted Message :")  
print(received.decode())
```

Output:

```
Sender :  
Enter Message :→HELLO RISHABH  
  
Ciphertext Sent (Hex):  
02742c7dfeed6add7abefff9ec51b4a8  
  
Receiver :→  
Received Ciphertext :  
02742c7dfeed6add7abefff9ec51b4a8  
Decrypted Message :  
HELLO RISHABH
```

Date

Marks

Signature

Practical No: 06

Problem Statement – 1

AIM: To implement the RSA algorithm for encrypting and decrypting a given plaintext using public and private keys.

Code:

```
# RSA Encryption and Decryption

import math

# Input
p = int(input("Enter prime number p: "))
q = int(input("Enter prime number q: "))
message = int(input("Enter message: "))

# Calculate n
n = p * q

# Calculate Euler's Totient
phi = (p - 1) * (q - 1)

# Find e
for e in range(2, phi):
    if math.gcd(e, phi) == 1:
        break

# Find d
for d in range(2, phi):
    if (d * e) % phi == 1:
        break

# Encryption
ciphertext = pow(message, e, n)

# Decryption
decrypted = pow(ciphertext, d, n)

# Output
print("\nPublic Key (e, n):", (e, n))
print("Private Key (d, n):", (d, n))
print("Ciphertext:", ciphertext)
print("Decrypted Text:", decrypted)
```

Output:

```
Enter prime number p: 4
Enter prime number q: 15
Enter message: 11

Public Key (e, n): (5, 60)
Private Key (d, n): (17, 60)
Ciphertext: 11
Decrypted Text: 11
```

Problem Statement - 2

Aim: To simulate secure communication between a sender and receiver using the RSA public-key cryptosystem.

Code:

```
# Secure Communication Using RSA

import math

# Receiver generates RSA keys
p = int(input("Enter prime number p: "))
q = int(input("Enter prime number q: "))
message = int(input("Enter secret message: "))

# Calculate n and phi
n = p * q
phi = (p - 1) * (q - 1)

# Generate public key e
for e in range(2, phi):
    if math.gcd(e, phi) == 1:
        break

# Generate private key d
for d in range(2, phi):
    if (d * e) % phi == 1:
        break

# Sender encrypts message using receiver's public key
encrypted_message = pow(message, e, n)

# Receiver decrypts using private key
decrypted_message = pow(encrypted_message, d, n)
```

```
# Display results
print("\nPublic Key:", (e, n))
print("Private Key:", (d, n))
print("Encrypted Message:", encrypted_message)
print("Decrypted Message:", decrypted_message)
```

Output:

```
Enter prime number p: 4
Enter prime number q: 15
Enter secret message: 11

Public Key: (5, 60)
Private Key: (17, 60)
Encrypted Message: 11
Decrypted Message: 11
```

Date

Marks

Signature

Practical No: 07

Case Study 1: Banking Transaction Authentication — MAC & Hash Functions

Q1. Identify the authentication requirements involved in this scenario.

The main authentication requirements are:

1. **Data Integrity** – The transaction amount and other details should not be modified.
2. **Message Authentication** – The bank should verify that the transaction comes from an authorized system.
3. **Protection against Tampering** – An attacker should not be able to change ₹50,000 to ₹5,00,000.
4. **Key Security** – The secret MAC key must remain known only to authorized parties.

Q2. Explain why a simple hash such as SHA-256 alone does not authenticate the sender.

- A simple hash such as SHA-256 only produces a fixed-size digest of the message.
- It can help detect whether data has changed, but it does not prove who created or sent the message.
- An attacker can modify the transaction and calculate a new SHA-256 hash because the hash does not require a secret key.
- Therefore, SHA-256 alone provides integrity checking but not sender authentication.

Q3. Explain how a MAC can protect the transaction.

A MAC (Message Authentication Code) uses:

- The transaction message
- A shared secret key

The sender generates a MAC using the message and secret key and sends the message + MAC to the bank.

The bank uses the same secret key to calculate the MAC again.

- If both MACs match → transaction is accepted.
- If they do not match → transaction is rejected.

Therefore, an attacker cannot easily modify the transaction without detection.

Q4. Explain the role of HMAC-SHA-256 and the secret key.

HMAC-SHA-256 is a MAC construction that uses the SHA-256 hash function together with a secret key.

The secret key is known only to authorized parties.

The sender uses the key to create the HMAC, and the receiver uses the same key to verify it.

Flow:

Message + Secret Key → HMAC-SHA-256 → MAC

The secret key makes it difficult for an attacker to create a valid MAC.

Q5. What happens when the transaction amount is modified?

Suppose the original transaction is:

₹50,000 → ₹5,00,000

Changing the amount changes the message. Therefore, when the bank recalculates the MAC using the secret key, the newly calculated MAC will be different from the original MAC. The comparison fails, so the bank rejects the transaction.

Q6. Why cannot an attacker simply calculate a new valid MAC without the secret key?

A MAC is generated using both the message and a secret key.

The attacker may know the message and the MAC algorithm, but does not know the secret key.

Without the secret key, the attacker cannot generate the correct MAC for the modified transaction.

Therefore, the modified transaction will fail MAC verification.

Case Study 2: Software Company — MD5, SHA & Birthday Attack

Q7. Explain the purpose of a cryptographic hash function.

A cryptographic hash function converts input data of any size into a fixed-size output called a hash value or digest.

Its main purpose is to:

- Detect changes in data.
- Verify data integrity.
- Create a unique-looking representation of the data.

Example:

File → SHA-256 → Hash/Digest

If even a small part of the file changes, its hash value will normally change.

Q8. List the important requirements of a secure hash function.

The three important requirements are:

1. **Preimage Resistance**
Given a hash value, it should be difficult to find the original message.
2. **Second-Preimage Resistance**
Given one message, it should be difficult to find another different message having the same hash.
3. **Collision Resistance**
It should be difficult to find any two different messages that produce the same hash.

Q9. Explain why MD5 is considered cryptographically broken for collision resistance.

MD5 produces a **128-bit hash**.

Cryptographic attacks have demonstrated practical methods for creating different inputs with the same MD5 hash.

This means MD5 no longer provides reliable collision resistance.

Therefore, MD5 is considered cryptographically broken and is unsuitable for security-sensitive applications such as digital signatures.

Q10. Explain the Birthday Attack in the context of hash collisions.

A Birthday Attack is an attack that tries to find two different messages having the same hash value.

For an **n-bit hash**, the generic collision search requires approximately:

$2^{(n/2)}$ attempts

This is much smaller than trying approximately **2^n** possible values.

For example, for SHA-256:

$2^{(256/2)} = 2^{128}$ attempts

Q11. Compare MD5, SHA-1, SHA-256, and SHA-512.

Algorithm	Digest Size	Collision Security
MD5	128 bits	Broken
SHA-1	160 bits	Collision attacks demonstrated
SHA-256	256 bits	Widely used
SHA-512	512 bits	Widely used

These values are given in the assignment's comparison table.

Q12. Explain why the generic birthday-search complexity for an n-bit hash is approximately $2^{(n/2)}$.

An n-bit hash has approximately **2^n possible hash values**.

Because of the Birthday Paradox, a collision becomes likely after testing roughly the square root of the possible hash values.

Therefore:

$\sqrt{2^n} = 2^{(n/2)}$

So the generic birthday-attack complexity is approximately:

$2^{(n/2)}$

Q13. For a 256-bit hash, calculate the approximate generic birthday-attack complexity.

For a 256-bit hash:

$n = 256$

Formula:

$2^{(n/2)}$

Therefore:

$2^{(256/2)}$

$= 2^{128}$ attempts

So the approximate generic birthday-attack complexity is:

2^{128} attempts

Case Study 3: Government Document Signing — Digital Signature & DSS**Q14. Explain why the document is hashed before signing.**

Instead of directly signing the entire document, the document is first passed through a hash function.

The hash function produces a fixed-size message digest.
The digest is then signed using the officer's private key.
This makes the signing process more efficient and allows the receiver to verify whether the document has changed.

Flow:

Document → Hash Function → Message Digest → Private Key → Digital Signature

Q15. Explain the difference between a private key, public key, and digital signature.

Term	Meaning
Private Key	Secret key owned by the signer and used to create the digital signature.
Public Key	Key shared with others and used to verify the signature.
Digital Signature	A cryptographic value created using the document digest and the signer's private key.

The private key must remain secret, while the public key can be shared.

Q16. Explain how digital signatures provide integrity and authentication, and how they support non-repudiation.

- **Integrity:**
If the document is modified after signing, its hash will change. Signature verification will therefore fail.
- **Authentication:**
The signature can be verified using the signer's public key, showing that the signature corresponds to the private key associated with the signer.
- **Non-repudiation:**
A digital signature can provide evidence linking the signature to the signer, helping prevent the signer from credibly denying having signed the document, assuming the private key was properly controlled and other relevant evidence supports the attribution.

Q17. Explain the role of DSS (Digital Signature Standard).

DSS (Digital Signature Standard) provides a standard framework for creating and verifying digital signatures.

In this case, DSS is used by the government department to digitally sign documents.

Basic process:

Document → Hash → Digest → Private Key + DSS → Digital Signature

The receiver uses the public key to verify the signature.

Q18. What happens if an attacker modifies an amount in the document after it has been signed?

Suppose the original document contains:

Amount = ₹50,000

The attacker changes it to:

Amount = ₹5,00,000

The modified document produces a different hash value.

During verification, the newly calculated digest will not match the digest represented by the valid signature.

Therefore:

Signature verification fails → Document is rejected as modified.

Q19. Why must the private signing key never be shared?

The private key is used to create digital signatures.

If an attacker obtains the private key, they may be able to create signatures that appear to come from the legitimate signer.

Therefore, the private key must:

- Remain secret.
- Be securely stored.
- Never be shared with unauthorized people.

Q20. Compare MAC and Digital Signature in terms of keys, authentication, integrity, non-repudiation, and verification.

Feature	MAC	Digital Signature
Keys	Uses a shared secret key	Uses private and public keys
Authentication	Authenticates a message between parties sharing the secret key	Can authenticate the signer through the public-key verification process
Integrity	Provides message integrity	Provides document/message integrity
Non-repudiation	Does not provide non-repudiation	Supports non-repudiation
Verification	Receiver uses the shared secret key	Receiver uses the signer's public key
Key Sharing	Secret key must be shared between authorized parties	Private key remains secret; public key can be shared

Date

Marks

Signature

Practical No: 08

Task 1 : Implementation of Diffie-Hellman Key Exchange using Python

Aim : To implement the **Diffie-Hellman Key Exchange algorithm** using Python and generate a common secret key between two users.

Theory

Diffie-Hellman Key Exchange is a cryptographic method used by two users to establish a **shared secret key** over an insecure communication channel.

In this method:

- Both users agree on a **prime number (p)** and a **primitive root (g)**.
- Alice and Bob choose their own **private keys**.
- They calculate their **public keys** using the private keys.
- Public keys are exchanged.
- Both users independently calculate the **shared secret key**.
- The shared secret key generated by Alice and Bob is the same.

Working

Public Values (p, g) → Alice (Private Key a) → Public Key A → Exchange Public Keys → Bob (Private Key b) → Public Key B → Shared Secret Key

Code:

```
# Diffie-Hellman Key Exchange

# Public values
p = int(input("Enter prime number (p): "))
g = int(input("Enter primitive root (g): "))

# Private keys
a = int(input("Enter private key of Alice: "))
b = int(input("Enter private key of Bob: "))

# Calculate public keys
A = pow(g, a, p)
B = pow(g, b, p)

# Calculate shared secret keys
alice_secret = pow(B, a, p)
bob_secret = pow(A, b, p)

# Display results
print("\nPublic Values:")
print("Prime (p):", p)
```

```

print("Primitive Root (g):", g)

print("\nPublic Keys:")
print("Alice Public Key:", A)
print("Bob Public Key:", B)

print("\nShared Secret Keys:")
print("Alice Secret Key:", alice_secret)
print("Bob Secret Key:", bob_secret)

# Verify shared secret
if alice_secret == bob_secret:
    print("Key Exchange Successful")
else:
    print("Key Exchange Failed")

```

Output:

```

Enter prime number (p): 15
Enter primitive root (g): 4
Enter private key of Alice: 20
Enter private key of Bob: 19

Public Values:
Prime (p): 15
Primitive Root (g): 4

Public Keys:
Alice Public Key: 1
Bob Public Key: 4

Shared Secret Keys:
Alice Secret Key: 1
Bob Secret Key: 1
Key Exchange Successful

```

Task 2 : Simulation of a Digital Signature using Python

Aim : To simulate the process of creating and verifying a **Digital Signature** using Python.

Theory

A **Digital Signature** is a cryptographic technique used to verify the **authenticity and integrity** of a message or document.

It helps the receiver verify that:

- The message was sent by the expected sender.
- The message has not been modified.
- The sender's signature can be verified using the corresponding verification information.

In a digital signature process, the sender creates a **hash** of the message and generates a signature using a **private key**. The receiver can verify the signature using the corresponding **public key**.

Working :

Message → Hash Generation → Digital Signature → Verification → Valid / Invalid

Code :

Digital Signature Simulation

```
import hashlib
import secrets
```

```
# Input message
message = input("Enter message: ")
```

```
# Generate private key
private_key = secrets.token_hex(16)
```

```
# Create hash of message
message_hash = hashlib.sha256(message.encode()).hexdigest()
```

```
# Create digital signature
signature = hashlib.sha256(
    (message_hash + private_key).encode()
).hexdigest()
```

```
print("\nOriginal Message:", message)
print("Message Hash:", message_hash)
print("Digital Signature:", signature)
```

```
# Verification
verification_hash = hashlib.sha256(
    (message_hash + private_key).encode()
).hexdigest()
```

```
if signature == verification_hash:
    print("\nDigital Signature Verification: SUCCESS")
else:
    print("\nDigital Signature Verification: FAILED")
```

Output:

```
Enter message: Hello Rishabh

Original Message: Hello Rishabh
Message Hash: ba47f48caf393db5fc7beccf6d90abce13b1f356c7f0ae6fffccdf2d629d395a
Digital Signature: 8f90d7c203ae4479069696662353e774b6cd9a4ad46cd8e4a4859d7fa4b10782

Digital Signature Verification: SUCCESS
```

Task 3 : Demonstrate Email Encryption using the PGP Tool

Aim : To demonstrate **email encryption and decryption using the PGP tool** for secure communication.

Theory

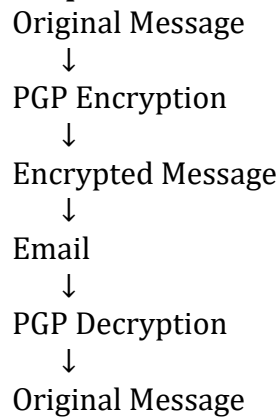
- **PGP (Pretty Good Privacy)** is a security tool used to protect emails and files. It uses encryption techniques to provide **confidentiality, authentication, integrity, and digital signatures**.
- PGP uses a **public key and private key**. The public key can be shared with others, while the private key must be kept secret.

Procedure / Steps

1. **Install a PGP Tool**
Install a PGP-compatible tool such as Gpg4win.
2. **Generate a Key Pair**
Create a new key pair containing:
 - Public Key
 - Private Key
3. **Export the Public Key**
Export the receiver's public key and share it with the sender.
4. **Import the Public Key**
The sender imports the receiver's public key into the PGP tool.
5. **Create the Email Message**
Write the confidential message that needs to be sent.
Hello,

This is a confidential message.
This message is encrypted using PGP.

Regards,Rishabh
6. **Encrypt the Message**
Select the message and choose the **Encrypt** option. Select the receiver's public key.
7. **Send the Encrypted Message**
Send the encrypted message through email.
8. **Receive the Encrypted Message**
The receiver receives the encrypted message.
9. **Decrypt the Message**
The receiver opens the PGP tool and selects the **Decrypt** option.
10. **Enter Private Key Password**
Enter the required password/passphrase for the private key.
11. **View the Original Message**
After successful decryption, the original readable message is displayed.

Output:

Result : Email encryption and decryption using the **PGP tool** was successfully demonstrated.

Task 4 : Case Study on Recent Cyberattacks and Prevention Techniques

Aim : To study a recent cyberattack, understand its method and impact, and identify suitable techniques for preventing such attacks.

Theory :

A **cyberattack** is an intentional attempt to gain unauthorized access to a computer system, network, application, or data. Cyberattacks can cause data loss, financial damage, service disruption, and privacy problems.

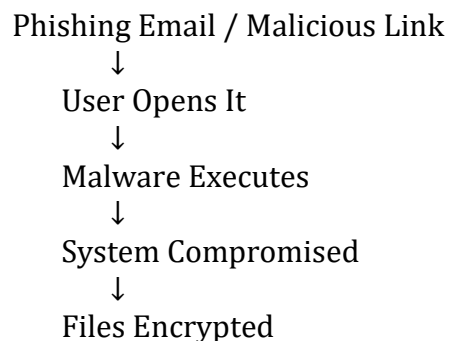
One common type of cyberattack is a **ransomware attack**.

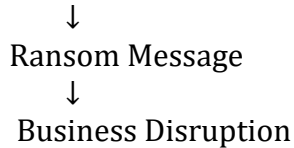
Ransomware is malicious software that can encrypt a victim's files and make them inaccessible. Attackers may then demand a ransom in exchange for restoring access.

Case Study: Ransomware Attack :

In a ransomware attack, attackers may first gain access through a **phishing email**, malicious attachment, compromised account, or vulnerable software.

After entering the system, the ransomware can spread to other systems and encrypt important files. The victim may then receive a ransom message demanding payment.

Attack Flow :



Impact of the Attack :

A ransomware attack can result in:

1. Loss of access to important files.
2. Disruption of normal business operations.
3. Financial losses.
4. Data recovery expenses.
5. Loss of customer trust.
6. Possible exposure or theft of sensitive information.

Prevention Techniques :

No.	Prevention Technique	Description
1	Regular Backups	Keep regular backups of important data and store them separately from the main system.
2	Employee Awareness	Train employees to identify phishing emails, suspicious links, and unknown attachments.
3	Antivirus Software	Use updated security software to detect and block malicious programs.
4	Software Updates	Keep operating systems and applications updated with security patches.
5	Strong Authentication	Use strong passwords and multi-factor authentication (MFA).
6	Access Control	Give users only the permissions required for their work.
7	Network Monitoring	Monitor systems and networks for unusual or suspicious activity.
8	Incident Response	Maintain a plan to isolate infected systems and recover affected data.

Prevention Flow :

Employee Awareness → Security Software → Software Updates → MFA → Access Control → Regular Backups → Monitoring → Incident Response

Case Study Summary :

Ransomware attacks can seriously affect organizations by making important data inaccessible and disrupting operations. Security awareness, regular backups, software updates, strong authentication, access control, monitoring, and an effective incident-response plan can help reduce the risk and impact of ransomware.

Result : The ransomware cyberattack case study was studied, and the major **prevention techniques** were identified.

Task 5 : Perform Basic Steganography using Open-Source Tools

Aim : To perform basic **image steganography** using an open-source tool and hide a secret message inside an image.

Theory

Steganography is a technique of hiding secret information inside another file such as an image, audio, or video.

In image steganography, a secret message is hidden inside a **cover image**. The resulting image is called a **stego image**.

The main purpose of steganography is to hide the **existence of the secret message**.

Working

Cover Image + Secret Message



Steganography Tool



Stego Image



Message Extraction



Secret Message

Procedure / Steps

1. Select a Cover Image

Choose an image that will be used to hide the secret message.

Example:

cover.png

2. Install/Open an Open-Source Steganography Tool

Open a suitable open-source steganography tool such as **OpenStego**.

3. Select Hide Data Option

Select the option to hide data inside an image.

4. Select the Cover Image

Choose the image file in which the secret message will be hidden.

5. Create the Secret Message

Create a text file containing the secret information.

Example:

This is my confidential message.

6. Select the Secret File

Select the text file as the data to be hidden.

7. **Select Output File**

Choose the location and filename for the output stego image.

Example:

stego-image.png

8. **Hide the Secret Message**

Start the hiding process. The tool embeds the secret data into the image.

9. **Extract the Hidden Message**

Use the extraction option and select the generated stego image.

10. **Verify the Message**

Extract the hidden data and open the resulting file to verify the original secret message.

Output :

Cover Image

+

Secret Message

↓

Steganography

↓

Stego Image

↓

Extraction

↓

Secret Message

Result :

Basic **image steganography** was successfully performed using an open-source tool, and the hidden secret message was successfully extracted.

Date

Marks

Signature